

Demonstrations are weights you write at inference time

And the non-negativity that makes those weights readable is what makes them forget.

DATAFORGE 2026, PATHWAY TRACK — ONE-PAGE CONCEPT SUMMARY. TOPIC: TEST-TIME ADAPTATION, OPTIMIZATION VERSUS CONTEXT.

The design pressure

A model meeting an unfamiliar task at inference has two routes. It can **optimize** — convert the demonstrations into training samples and take a backward pass before answering, as HRM (arXiv:2506.21734) and TRM (arXiv:2510.04871) do on ARC at roughly \$1.48–1.76 per task. Or it can **adapt in state**: read the demonstrations, write them into a recurrent memory, answer with every parameter untouched. Pathway's BDH-CQ takes the second route at three orders of magnitude less. The interesting question is not which is cheaper, but what the second route *charges* instead.

The mechanism, and the part everyone guesses wrong

A softmax-free attention with $Q = K$ has an exact recurrent form. Where a Transformer keeps a KV cache that grows with every token, the same computation can be carried in a fixed-size associative state updated by rank-one writes:

$$\begin{aligned}\sigma_i &= \sigma_{i-1} + \text{rope}(K)_i \otimes V_i \\ \text{out}_i &= \sigma_i^\top \text{rope}(Q)_i\end{aligned}$$

Not an approximation: the official MIT `bdh.py` run both ways agrees to 2.8×10^{-14} in float64, the same arithmetic reassociated. The natural assumption is that BDH's three unusual choices — no softmax, $Q = K$, and a ReLU forcing activations non-negative — all buy that constant memory. **They do not.** Ablating each: adding a

softmax breaks the equivalence (1.2×10^2), while $Q \neq K$ (1.7×10^{-5}) and dropping the ReLU (1.6×10^{-5}) leave it intact. Softmax-freeness buys the fixed-size state. $Q = K$ and the ReLU buy something else: *readability*.

What readability costs

Non-negative activations are what make BDH inspectable — sparse, positive, monosemantic structure is the Dragon Hatchling paper's claim (arXiv:2509.26507). On an 8M BDH we trained we measure 5.11% of neurons active, and a standing synapse graph $G^* = D_x^\top E^\top$ that predicts **from the weights alone, with no data**, which neurons stay silent on real text (Matthews correlation +0.944).

That same non-negativity caps what the state can hold. Writing k rank-one bindings into a fixed $N \times D$ state and reading them back, retrieval is 100% at $k=8$, 94% at $k=32$, and 47% at $k=128$. The counterfactual identifies the cause: with **signed** keys the identical state holds 384 bindings at 99% — past its own rank. Mean pairwise cosine is **+0.319 for ReLU'd keys versus -0.000 for signed**, and +0.319 is not empirical noise but analytic: $1/\pi$ for rectified Gaussians. Non-negative vectors cannot be near-orthogonal, so they interfere. Interpretability is paid for in memory, at a fixed exchange rate.

The ceiling is known to the field, which is the point: DeltaNet (arXiv:2406.06484) exists because "the additive update in linear transformers" — precisely BDH's write — is weak at associative recall. Ours is the mechanism and the constant.

	KV-cache Transformer	BDH (public, MIT)	BDH-CQ	HRM / TRM
Memory per extra token	grows linearly	constant	constant (dims undisclosed)	no context state
Adaptation lives in	the context window	recurrent state σ	recurrent state	weights — a backward pass
Inspectability	attention maps	sparse activations + synapse graph	none published	none
Evidence here	—	we run it	replayed, cited	reported by authors

The roles of BDH and BDH-CQ, stated precisely

They are not interchangeable. **BDH is the public substrate** — MIT-licensed, runnable, and what every number above was measured on. **BDH-CQ is evidence only**: no public weights, no API, and the report states dimensions and update rules "remain proprietary."

The bridge is exact and citable. BDH-CQ's memory update is Eq. (1) $S_i = U_\theta(S_{i-1}, D_i)$, and §3.2 names its special case $S_i = S_{i-1} + U_\theta(D_i)$ as "linear attention being the conceptually simplest standalone realization." **That special case is exactly what `bdh.py` computes.** One correction matters: additive accumulation is the *named special case*, not BDH-CQ's rule — describing it as BDH-CQ's mechanism, as secondary summaries do, is an error.

Reading the evidence honestly

BDH-CQ's headline is 29.5% pass@2 (118/400) on public ARC-AGI-1 at ~0.85 H200 GPU-seconds/task (arXiv:2608.09888, Table 1). Four qualifications travel with it, all from the report itself:

- \$0.00070/task is computed, not billed** — 0.85 s $\times$ an assumed \$3/H200-hour. §6.6 reports the *same* 118/400 at **\$0.00265246**, 3.8 $\times$ higher, and never reconciles the two. At that figure "57 $\times$ cheaper" becomes ~15 $\times$.
- The comparison is **not equal-accuracy**: GPT-5.6 Luna (Low) scores *higher*, 34.2%. The Wilson interval on 29.5% is [25.24, 34.15] — it overlaps.

- The independent audit reproducing 29.5% was conducted by **co-authors**. A developer-reported result is not an external reproduction, and BDH-CQ appears on no ARC Prize leaderboard.
- ConceptARC is in the training mixture** (§4.2) and then used for evaluation (59.38%). The paper says its control does not rule out training exposure.

The sharpest limitation is structural, not numerical. Latent reasoning returns grids, not traces, so — the report's own words — "a correct rule applied incompletely is observationally indistinguishable from a narrower rule applied completely." Table 9 makes it concrete: 89.7% of failures still have correct output dimensions. Not verbalizing is what makes it cheap, and why you cannot audit it. Ours mirrors it in miniature: a 131K toy and an 8M translation model are honest instruments, not evidence about BDH at scale.

Continue at the artifact — <https://demonstrations-are-weights.vercel.app> — then the Dragon Hatchling paper, BDH-CQ §3.2 and Appendix A.3–A.4, and DeltaNet for where the field went next.

Provenance. Every figure attributed to us is reproducible from github.com/snivellus38/DataForge by a named script; every BDH-CQ figure carries a section or table locator into arXiv:2608.09888 and is never recomputed here. `vendor/bdh.py` is Pathway's, MIT and unmodified. Our models are a 131K BDH trained for this submission and an 8M BDH trained by one author beforehand and reused with disclosure (it adds a learned positional embedding over RoPE and has no decay term). Neither is an official BDH model; neither is BDH-CQ. AI-assisted, disclosed in the README.